

Tarea 07 — Secciones 4.1 y 4.2

Investigación de herramientas de prueba y análisis estático

Estándares y Métricas para el Desarrollo de Software (E-EMDS-2)

4. Desarrollo

4.1. Investigación: frameworks de automatización E2E

La automatización de pruebas de extremo a extremo (*end-to-end*, E2E) valida el comportamiento del sistema desde la perspectiva del usuario final, ejercitando la interfaz real contra una aplicación en ejecución (Jorgensen, 2022). Para nuestro proyecto integrador —un wiki colaborativo construido con un frontend en React/TypeScript y un backend en Node.js— la elección del framework determina la velocidad de la retroalimentación, la estabilidad de la *suite* y el costo de mantenimiento a largo plazo. A continuación comparamos las tres herramientas de referencia del ecosistema E2E en 2026: **Selenium WebDriver**, **Cypress** y **Playwright**.

Selenium WebDriver. Es la herramienta más madura del ecosistema y opera sobre el estándar *W3C WebDriver*, comunicándose con cada navegador a través de su *driver* correspondiente (Selenium Project, 2026). Su mayor fortaleza es la amplitud: soporta múltiples lenguajes (Java, Python, C#, JavaScript, Ruby, Kotlin) y la mayor variedad de navegadores, incluyendo motores heredados. La gestión de esperas se realiza mediante esperas explícitas con `WebDriverWait` y condiciones esperadas, lo que evita la fragilidad de las esperas fijas (García, 2022). Como contrapartida, exige más código de andamiaje (*boilerplate*), su curva de aprendizaje es más pronunciada y la ejecución en paralelo depende de infraestructura adicional (Selenium Grid).

Cypress. Adopta una arquitectura distinta: ejecuta el código de prueba *dentro* del propio navegador, en el mismo bucle de eventos de JavaScript que la aplicación bajo prueba (Cypress.io, 2026). Esto le otorga latencia de red nula, espera automática (*auto-waiting*) y un depurador con “viaje en el tiempo” que captura el estado del DOM en cada paso, lo que se traduce en una experiencia de desarrollo (DX) superior. Sus limitaciones son consecuencia de esa misma arquitectura: solo admite JavaScript/TypeScript, su soporte de WebKit/Safari es experimental e incompleto, y la ejecución paralela robusta requiere el servicio de pago Cypress Cloud.

Playwright. Desarrollado por Microsoft, automatiza de forma nativa los tres motores principales —Chromium, Firefox y WebKit— y ofrece enlaces oficiales para JavaScript/TypeScript, Python, Java y C# (Microsoft, 2026). Incorpora espera automática basada en comprobaciones de *actionability* (el elemento debe ser visible, estable, habilitado y no estar obstruido antes de interactuar), contextos de navegador aislados, un *Trace Viewer* para el análisis post-mortem y, de forma destacada, ejecución paralela y *sharding* nativos sin necesidad de un *grid*. Estas características reducen la cantidad de pruebas inestables (*flaky*) y disminuyen el costo total de propiedad al escalar la *suite*.

Tabla comparativa. La Tabla 1 sintetiza la comparación según diez criterios técnicos relevantes para nuestro contexto.

Tabla 1: Comparación de frameworks de automatización E2E (2026).

Criterio	Selenium	Cypress	Playwright
Lenguajes soportados	Java, Python, C#, JavaScript, Ruby, Kotlin	Solo JavaScript / TypeScript	JavaScript/TypeScript, Python, Java, C#
Arquitectura	Protocolo W3C WebDriver vía <i>drivers</i> externos	Ejecuta dentro del navegador (mismo bucle de JS)	Comunicación directa con el navegador (Web-Socket/CDP)
Curva de aprendizaje	Pronunciada; requiere más <i>boilerplate</i>	Baja para equipos de JS; DX muy amigable	Media; API moderna y <i>codegen</i>
Velocidad / rendimiento	Moderada; sobrecarga de WebDriver	Buena en local; arranque más lento	Alta; la más rápida en ejecución paralela
Manejo de esperas	Explícitas (<code>WebDriverWait</code>); evita esperas fijas	Espera automática integrada	Espera automática por <i>actionability</i>
Soporte de navegadores	El más amplio (incluye motores heredados)	Chromium y Firefox nativos; WebKit experimental	Chromium, Firefox y WebKit nativos
Ejecución en paralelo	Mediante Selenium Grid (infraestructura adicional)	Limitada; paralelismo robusto requiere Cypress Cloud	Nativa, sin <i>grid</i> (<i>workers</i> y <i>sharding</i>)
Pruebas móviles	Nativas vía Appium	No (solo emulación de <i>viewport</i>)	No (solo emulación de dispositivos)
Comunidad / madurez	La más madura; estándar de facto empresarial	Fuerte en <i>frontend</i> JS; comunidad consolidada	Crecimiento más rápido; comunidad en expansión
Licencia	Apache 2.0 (código abierto)	MIT (código abierto; <i>Cloud</i> de pago)	Apache 2.0 (código abierto)

Fuente: elaboración propia con base en la documentación oficial de Selenium, Cypress y Playwright (Selenium Project, 2026; Cypress.io, 2026; Microsoft, 2026) y en García (2022).

Recomendación justificada. Para el contexto de nuestro proyecto —una aplicación web de página única (SPA) con uso intensivo de JavaScript y un *stack* de TypeScript/React/Node.js— la recomendación técnica es **Playwright**. Tres razones lo respaldan: (1) sus enlaces de

primera clase para TypeScript se alinean directamente con el lenguaje del proyecto, reduciendo la fricción de adopción; (2) su soporte nativo de WebKit permite cubrir Safari, navegador relevante para un wiki de acceso público que Cypress no cubre de forma fiable; y (3) su ejecución paralela nativa, junto con la espera por *actionability*, ataca directamente el principal problema de las *suites* E2E —la inestabilidad— sin costos de infraestructura adicionales, en línea con el principio de pruebas rápidas y confiables que recomienda Aniche (2022).

No obstante, conforme a lo trabajado en la Semana 7, la *implementación* práctica (sección 4.3 y siguientes) se realiza con **Selenium** + **pytest** aplicando el patrón *Page Object Model*. Esta diferencia entre la herramienta recomendada y la implementada es deliberada y está prevista por la asignatura: Selenium ofrece el soporte más amplio de lenguajes y navegadores, una base conceptual sólida sobre el estándar WebDriver y un ecosistema empresarial maduro, lo que lo hace idóneo como herramienta de aprendizaje (García, 2022). Las técnicas de diseño de casos (válidos, de frontera y de error) que aplicamos son independientes del framework (Jorgensen, 2022), por lo que la suite construida con Selenium puede migrarse a Playwright en el futuro conservando su valor.

4.2. Investigación: análisis estático de código

El análisis estático examina el código fuente *sin ejecutarlo*, identificando defectos a partir de su estructura sintáctica y semántica. Es una práctica de “desplazamiento a la izquierda” (*shift-left*) que detecta problemas en las etapas tempranas del ciclo de vida, cuando corregirlos es más barato. En esta sección comparamos tres herramientas representativas de categorías distintas —**SonarQube**, **Semgrep** y los *linters* **ESLint**/**Pylint**— y diferenciamos el análisis estático del dinámico.

SonarQube. Es una plataforma de calidad y seguridad de código que cubre más de 30 lenguajes mediante miles de reglas predefinidas (SonarSource, 2026). Detecta *bugs* (confiabilidad), vulnerabilidades y *security hotspots* (seguridad) y *code smells* (mantenibilidad), y además mide cobertura, duplicación y complejidad ciclomática y cognitiva. Su característica distintiva son las *Quality Gates*: criterios automáticos de aprobación o rechazo que pueden bloquear la fusión (*merge*) de una rama cuando el código nuevo no cumple los umbrales definidos. Es importante una precisión para la sección 4.4: bajo el modelo *Clean Code* actual, SonarQube define *tres* cualidades de software —Seguridad, Confiabilidad y Mantenibilidad— que, sumadas a las medidas de *Cobertura* y *Duplicación* del tablero, conforman las cinco dimensiones que reporta la herramienta (SonarSource, 2026).

Semgrep. Es un motor de análisis estático orientado a la seguridad (SAST) que opera mediante coincidencia de patrones sobre el árbol de sintaxis abstracta, no sobre texto plano. Su fortaleza es la rapidez (escaneos del orden de segundos) y la facilidad para escribir reglas personalizadas en YAML, lo que lo hace popular para hacer cumplir políticas de seguridad específicas. Detecta patrones explotables —inyección SQL, *cross-site scripting* (XSS), credenciales codificadas, deserialización insegura— mediante análisis de contaminación (*taint analysis*). Su limitación es que está enfocado en seguridad: no calcula métricas de mantenibilidad, complejidad, duplicación ni cobertura.

ESLint / Pylint (linters). Representan la capa más rápida y cercana al desarrollador. ESLint analiza JavaScript y TypeScript mediante un sistema de reglas configurable basado en el AST, mientras que Pylint hace lo equivalente para Python. Aplican estilo y convenciones, detectan errores comunes y se ejecutan en tiempo real dentro del IDE. Su limitación es la profundidad: no realizan análisis de seguridad entre archivos ni detectan errores lógicos o de tiempo de ejecución; por ello suelen complementarse con una herramienta SAST o una plataforma de calidad.

Tabla comparativa. La Tabla 2 resume las diferencias. Se incluye CodeClimate (hoy en transición a “qlty”) como referencia adicional de plataforma orientada a la mantenibilidad.

Tabla 2: Comparación de herramientas de análisis estático (2026).

Criterio	SonarQube	Semgrep	ESLint / Pylint
Categoría	Plataforma de calidad + seguridad	Motor SAST (seguridad)	<i>Linter</i> (estilo + errores básicos)
Defectos que detecta	<i>Bugs</i> , vulnerabilidades, <i>code smells</i> , duplicación	Vulnerabilidades de seguridad y <i>anti-patrones</i>	Estilo, convenciones y errores comunes
Enfoque de detección	Miles de reglas predefinidas por lenguaje	Coincidencia de patrones sobre el AST (reglas en YAML)	Reglas configurables sobre el AST
Métricas de calidad	Sí: complejidad, duplicación, cobertura, deuda técnica	No (solo seguridad)	No (solo reglas de estilo/sintaxis)
Integración CI/CD	<i>Quality Gates</i> que bloquean <i>merges</i>	Rápida; ideal para escaneo en cada <i>pull request</i>	En cada <i>commit</i> ; muy rápida
Lenguajes	30+ lenguajes	35+ lenguajes	ESLint: JS/TS; Pylint: Python
Licencia / costo	<i>Community Edition</i> gratuita; ediciones de pago	Núcleo de código abierto (LGPL); gratis hasta cierto número de contribuyentes	Código abierto (gratuito)

Fuente: elaboración propia con base en la documentación oficial de las herramientas (SonarSource, 2026).

Análisis estático frente a análisis dinámico. La diferencia fundamental es la *ejecución*. El análisis estático inspecciona el código sin correrlo y, por ello, puede razonar sobre *todas* las rutas del programa, detectando *code smells*, vulnerabilidades por patrón, complejidad excesiva y duplicación antes de que el software se ejecute siquiera. Su contrapartida son los falsos positivos y la imposibilidad de observar comportamientos que solo emergen en tiempo de ejecución. El análisis dinámico, en cambio, examina el programa *mientras se ejecuta* con entradas concretas: detecta fallos reales, fugas de memoria, cuellos de botella de rendimiento y condiciones de carrera, pero solo sobre las rutas efectivamente ejercitadas (de ahí su riesgo de falsos negativos) y requiere un entorno de ejecución y datos de prueba. De hecho, la propia *suite* E2E con Selenium de la sección 4.3 constituye una forma de análisis dinámico, pues valida la aplicación en funcionamiento. Ambos enfoques son complementarios y se integran en el Plan de Pruebas de la Unidad II: SonarQube (estático) y la suite de pruebas con pytest/Selenium (dinámico) cubren clases de defectos distintas (Aniche, 2022; Khorikov, 2020).

Justificación para nuestro *stack*. Recomendamos **SonarQube** como herramienta principal de análisis estático para el proyecto por cuatro motivos: (1) analiza de forma nativa JavaScript y TypeScript, los lenguajes de nuestro frontend y backend; (2) unifica en un solo tablero las cinco dimensiones que pide la evaluación (Confiabilidad, Seguridad, Mantenibilidad, Cobertura y Duplicación), facilitando la sección 4.4; (3) sus *Quality Gates* se integran con el pipeline de CI de la sección 4.5 para bloquear fusiones que no cumplan los umbrales; y (4) su *Community Edition* es gratuita y, mediante SonarCloud, el análisis es gratuito para repositorios públicos. Como complemento, conservamos **ESLint** —ya estándar en proyectos de React/Node.js— como capa rápida de *linting* en cada *commit*, y dejamos abierta la incorporación de **Semgrep** si el equipo requiere profundizar en seguridad. Esta estratificación (*linter* en cada *commit*, plataforma de calidad en cada PR) es la práctica recomendada para mantener la salud del código sin sacrificar velocidad (Aniche, 2022).

Referencias

- Aniche, M. (2022). *Effective software testing: A developer's guide*. Manning Publications.
- Cypress.io. (2026). *Cypress documentation*. <https://docs.cypress.io/>
- García, B. (2022). *Hands-on Selenium WebDriver with Java*. O'Reilly Media.
- International Software Testing Qualifications Board. (2023). *Certified tester foundation level syllabus v4.0*. ISTQB. <https://www.istqb.org/>
- Jorgensen, P. C. (2022). *Software testing: A craftsman's approach* (5.^a ed.). CRC Press.

Khorikov, V. (2020). *Unit testing principles, practices, and patterns*. Manning Publications.

Microsoft. (2026). *Playwright documentation*. <https://playwright.dev/>

Selenium Project. (2026). *Selenium documentation*. <https://www.selenium.dev/documentation/>

SonarSource. (2026). *SonarQube documentation*. <https://docs.sonarsource.com/>